

Padrão Arquitetural Portal QA*

[Visão Geral](#)

[Estrutura da Solução \(Backend\)](#)

[Estrutura Frontend na Solução](#)

[Estratégia de Testes](#)

Visão Geral

O projeto **Portal QA** foi estruturado seguindo os princípios da **Clean Architecture** (Arquitetura Limpa), orientada pelo **Domain-Driven Design (DDD)**.

A escolha deste padrão baseia-se em dois pilares principais:

Alinhamento Estratégico: Segue o padrão de desenvolvimento consolidado e adotado pela Autoglass, garantindo que qualquer desenvolvedor da organização consiga dar manutenção no código sem curva de aprendizado excessiva.

Qualidade de Software: Como uma ferramenta de Qualidade, o Portal QA deve ser exemplo de testabilidade e desacoplamento.

Principais Benefícios desta Estrutura:

Independência de Frameworks: O "coração" do negócio (Domínio) não depende do banco de dados, da UI ou de bibliotecas externas.

Testabilidade: A separação de responsabilidades facilita a aplicação de **TDD (Test-Driven Development)**, permitindo testes de unidade isolados nas regras de negócio.

Manutenibilidade: Mudanças no banco de dados ou na interface do usuário não quebram as regras de negócio.

Estrutura da Solução (Backend)

A solução está dividida em camadas concêntricas, onde a dependência aponta sempre para dentro (em direção ao Domínio).

Autoglass.PortalQA.Dominio (O Coração)

É a camada mais importante. Ela ignora totalmente a existência de banco de dados ou API.

O que contém: Entidades (**Entities**), Objetos de Valor (**ValueObjects**), Enums, Exceções de Domínio e Contratos (Interfaces) para repositórios.

Objetivo: Garantir que as regras de negócio do Portal QA sejam puras e imutáveis por fatores externos.

Autoglass.PortalQA.Aplicacao (A Orquestração)

Responsável por coordenar o que o sistema faz.

O que contém: Casos de uso implementados via padrão **CQRS** (Separation of Command and Query), divididos em **Commands** (escrita) e **Queries** (leitura), além de DTOs e validações.

Objetivo: Receber uma requisição da API, orquestrar as regras do Domínio e salvar/recuperar dados usando as interfaces.

Autoglass.PortalQA.Infraestrutura (O Encanamento)

Responsável por tudo que é externo ao código C# puro (Banco de Dados, Integrações, Arquivos).

O que contém: Implementação do acesso a dados (**Data**), mapeamento do Entity Framework (**Configurations**), e interceptadores.

Objetivo: Implementar as interfaces definidas no Domínio. Aqui reside a conexão concreta com o **MySQL**.

Autoglass.PortalQA.API (A Porta de Entrada)

É a interface de comunicação com o mundo externo (Frontend).

O que contém: **Controllers** , Filtros de Exceção e Injeção de Dependência.

Objetivo: Expor os endpoints REST que serão consumidos pelo Angular.

Estrutura Frontend na Solução

Autoglass.PortalQA.WEB (Angular)

Diferente de abordagens onde o Frontend fica em um repositório isolado, optou-se por manter o projeto **Angular** dentro da mesma *Solution* (.sln).

Justificativa Técnica:

Coesão do Produto: Trata-se de uma aplicação monolítica modular. Manter o Frontend junto ao Backend facilita a visão sistêmica do projeto, ideal para o perfil **Full Stack**.

DevOps Simplificado: Facilita a configuração de pipelines de CI/CD (Jenkins/Azure DevOps), permitindo que o *build* e *deploy* da aplicação completa sejam feitos a partir de um único versionamento.

Desenvolvimento Ágil: Permite rodar a solução completa (API + SPA) com um único comando de *start* durante o desenvolvimento local, agilizando o ciclo de feedback.

Estratégia de Testes

Para garantir a confiabilidade condizente com o time de Qualidade, a estrutura conta com uma pasta dedicada `tests/` que espelha a arquitetura:

UnitTests: Focados no Domínio e Aplicação (Validar regras sem banco de dados).

IntegrationTests: Focados na Infraestrutura e API (Validar persistência no MySQL e contratos HTTP).